

AI651

Assignment 1

Roll number: 25280016 Name: Abdul Sammad

5 October 2026

GitHub repository: <https://github.com/AbdulSammad1/Deep-Learning-For-Space-Time-and-Graph-Assignment-1>

Response 1A

Question: Before Output 1.1, record the expected order of the four forecasters for stations 1–3 and for held-out station 4. Then compare both station groups and Products A, B, and C, quantify the gaps, revise the order if needed, and explain the gaps from each model’s assumptions.

Answer: Before running the code, I was expecting Period-routed ridge to be the best model. After that I was expecting Raw Attention, and then Shared ridge and Sensor-specific ridge. For station 4, Sensor-specific ridge cannot be used because this station was not present in training.

So after running the models, the result was basically the same as I expected. The RMSE values are shown below. Here smaller RMSE means better prediction.

Model	Stations 1–3 RMSE	Station 4 RMSE	Parameters	Fit time (s)
Period-routed ridge	0.166	0.173	59,904	0.034
Raw Attention	0.438	0.637	368,368	75.092
Shared ridge	0.767	0.876	4,608	0.030
Sensor-specific ridge	0.768	Not available	13,824	0.109

Table 1: Output 1.1 validation RMSE.

If we look at Products A, B and C separately, Period-routed ridge is still best. For stations 1–3 its errors are 0.181 / 0.164 / 0.152. Raw Attention gives 0.422 / 0.474 / 0.415, while Shared ridge gives 0.805 / 0.854 / 0.621. For station 4, Period-routed ridge gives 0.184 / 0.182 / 0.150 and Raw Attention gives 0.646 / 0.599 / 0.663.

So the gap is quite clear. On stations 1–3, Raw Attention has about 2.6 times the error of Period-routed ridge. Shared ridge is about 4.6 times worse. Sensor-specific ridge is almost the same as Shared ridge, 0.768 compared with 0.767.

The reason makes sense to me. Shared ridge is using one fixed rule for every window, but the products are not running at the same pace. Sensor-specific ridge makes a different rule for each station, but the pace can still change inside that station. Period-routed ridge first checks the pace and then uses the matching rule, so it has an advantage. Raw Attention can change the weights from one window to another, so it does better than the simple ridge models, but here it still does not beat the model which is directly using pace information.

Response 1B

Question: Use Output 1.2 to show why one fixed map must compromise when the same station runs at two paces, and point to the phase or shape. Use Output 1.3 to show how the same learned projections can still produce different attention weights. Do not claim that attention recovered the physical period.

Answer: So here I am checking the same station but with two different product speeds. Product A has period 16.135 samples and Product C has period 30.093 samples. Shared ridge gives RMSE 0.779 for the fast Product A run and 0.448 for the slower Product C run.

From the graph, on Product A the Shared ridge prediction is too smooth and some peaks come at the wrong position. On Product C it looks better because the wave is slower, but the peak shape is still a bit flat. Period-routed ridge gives much smaller errors, 0.232 and 0.150. So one fixed matrix is trying to handle two quite different cycle lengths, and that is where it has to compromise.

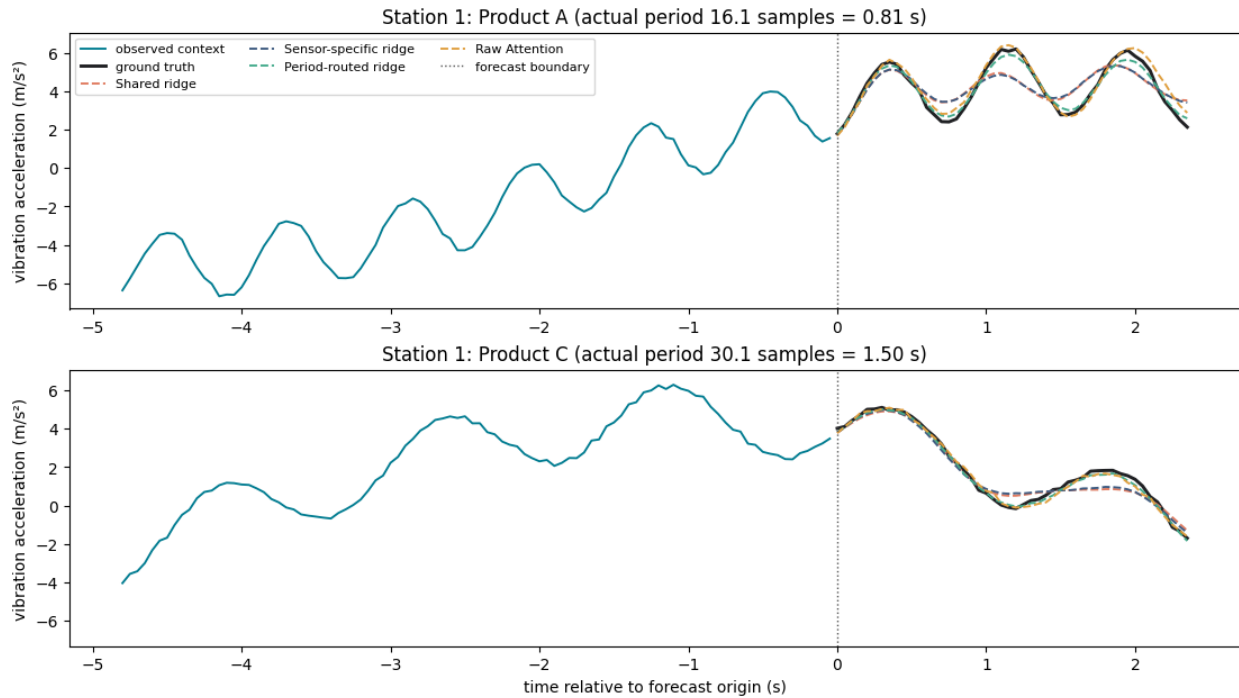


Figure 1: Output 1.2. Station 1 with Product A and Product C.

Output 1.3 is a different check. The learned projections are the same, but the attention weights made from the two windows are different. The mean absolute difference is 0.012 and the largest difference is 0.454. So changing the input can change the attention pattern even when the learned projections stay the same. I am not saying attention found the real 16-sample or 30-sample physical period. This figure only shows that the weights changed.

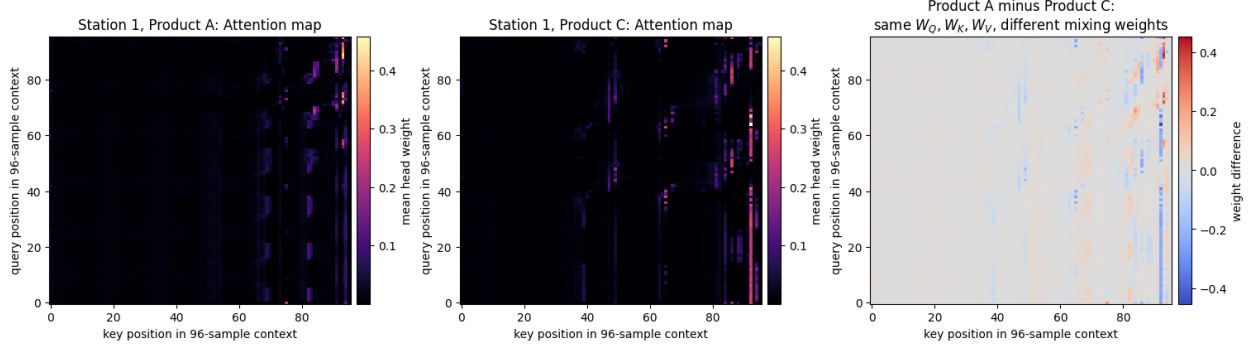


Figure 2: Output 1.3. Attention weights for the two products.

Response 2

Question: State the selected moving-average width. Describe what changes in Outputs 2.1–2.3, including the validation comparison of Raw Attention and Attention plus decomposition. Explain why a moving average fits this drift-plus-vibration signal, and why the training-only period check is a fair way to choose the width. Give one case where this bias fails, or name another decomposition and the assumption it would make.

Answer: So I selected moving-average width 41.

In Output 2.1, I tried different widths and checked what happens to the trend and leftover signal. With a small width, the average also follows some of the small vibration, so part of the vibration gets mixed into the trend. At width 17, the leftover standard deviation is 0.578. At width 41, the slow ramp stays more in the trend and the repeating wave is clearer in the leftover, so the leftover standard deviation becomes 1.572.

The Product C example in the figure has a true period of 32 samples. With width 41, the leftover gives a clear peak around this value, while the raw signal does not show it clearly. So in this example, width 41 is doing a better job of separating the slow part and the repeating part.

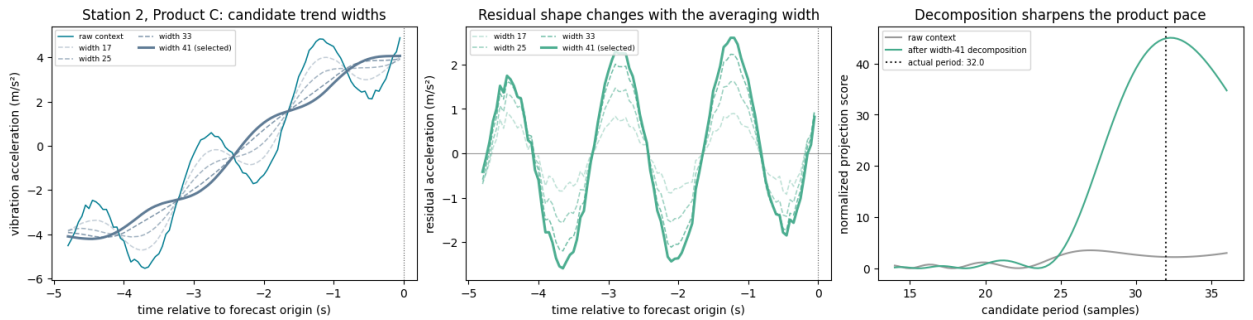


Figure 3: Output 2.1. Different moving-average widths.

Output 2.2 was only used for selecting the width. It checks how often the recovered period is within one sample of the real period. The raw signal gets 0.687. The scores for widths 17, 25, 33 and 41 are 0.980, 0.989, 0.993 and 0.998. So I picked 41 because it has the highest recovery score. Its median period error is 0.216 samples. Width 25 has a smaller median error of 0.122, but that was not the rule I was using for selection.

Also, this period check is only done on training data for choosing the width. The true period is not passed to the forecaster when it makes predictions, so I am not giving the model the answer from validation or test data.

For Output 2.3, the useful point is the product-by-product result. On stations 1–3, Raw Attention gives 0.422 / 0.474 / 0.415 for Products A / B / C, and Attention plus decomposition gives 0.284 / 0.337 / 0.287. So it improves all three products. The overall RMSE comparison is shown later in Output 4.1: 0.438 to 0.304 for stations 1–3, and 0.637 to 0.457 for station 4.

A moving average fits this signal because the drift is slow, around 240 samples, while the vibration repeats much faster, around 14 to 36 samples. So width 41 can mostly follow the slow movement and leave the faster repeating part in the leftover. Then the encoder can focus more on that repeating part, while the linear head handles the slow trend.

This idea can fail if the signal suddenly jumps. A moving average can spread that jump over many samples and make it look like part of the vibration. Another option is a Hodrick–Prescott smoother, but that method is also making an assumption: it expects the trend to change smoothly.

Response 3

Question: Confirm the delay-score check and the aggregation example. From Output 3.2, which lags repeat most strongly, and how do they relate to the run’s period? Why is the leftover cleaner than the raw signal, and why does that support delay mixing? Describe one signal for which a few repeating delays would be the wrong bias. Do not call the figure a plot of the trained model’s scores.

Answer: So first I checked the small series [1, 3, 1, 3]. The FFT delay scores are [4, −4, 4, −4], and direct circular correlation gives the same values. Then for the aggregation example, the values are [10, 20, 30, 40], delays are 1 and 3, and weights are 0.75 and 0.25. The result becomes [35, 15, 25, 25]. So the delay is reading from earlier positions in the sequence.

In Output 3.2, station 2 is on Product A and its true period is 16.135 samples. The strongest leftover correlations are 0.896 at delay 16 and 0.897 at delay 32. So these are almost one period and two periods. The raw signal does not show these peaks clearly. It mostly gives one wide hump because the rising drift still looks similar even after shifting it.

After removing the drift, the repeating part becomes much easier to see at the correct spacing. That is why doing decomposition before delay mixing makes sense here. One important thing is that this figure is showing correlation of the signal itself. It is not a plot of the trained model’s learned delay scores.

Delay mixing would be a bad choice for something like one single spike, because there is no real repeating pattern to copy. It can also struggle if the speed keeps changing inside one 96-sample window, because a few fixed delays may keep repeating the wrong spacing.

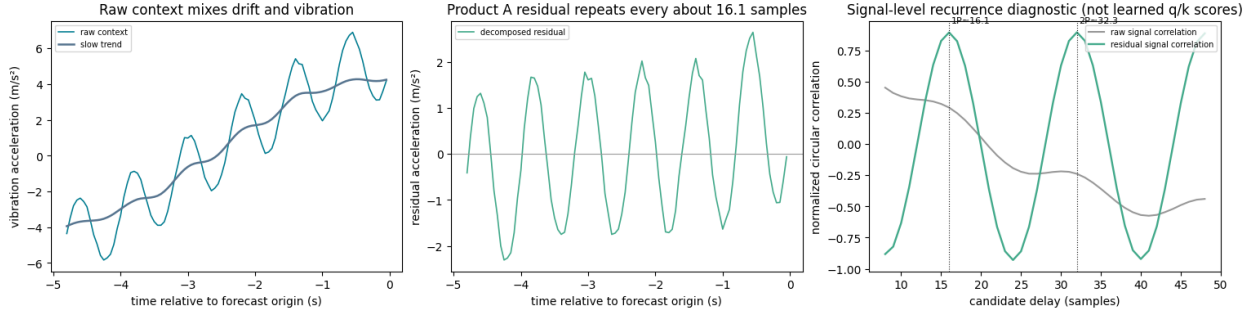


Figure 4: Output 3.2. Correlation of the signal, not trained model scores.

Response 4

Question: (a) In at most two sentences, before Output 4.1, say which upgrade should help most when the slow part is large, and why. (b) Report the validation RMSEs and MSEs, compare the two architectural switches, and summarize how error changes across the 48-step horizon. (c) Choose the deployment model for stations 1–3 and for station 4 using validation error, parameter count, and fit time.

Answer: Before seeing Output 4.1, I thought decomposition should help most when the slow part is large. My reason was simple: it removes the slow trend first, while delay mixing is mainly trying to use the repeating part.

Model	St. 1–3 RMSE	St. 1–3 MSE	St. 4 RMSE	St. 4 MSE
Period-routed ridge	0.166	0.028	0.173	0.030
Autoformer-inspired	0.198	0.039	0.242	0.059
Raw delay mixer	0.224	0.050	0.331	0.109
Attention + decomposition	0.304	0.092	0.457	0.209
Raw Attention	0.438	0.192	0.637	0.405
Shared ridge	0.767	0.588	0.876	0.767
Sensor-specific ridge	0.768	0.590	Not available	Not available

Table 2: Validation results for the fitted models.

After looking at the result, delay mixing alone actually gives the bigger single improvement. For stations 1–3, Raw Attention is 0.438 and Raw delay mixer is 0.224, so the improvement is 0.214. Attention plus decomposition is 0.304, so decomposition alone improves it by 0.134. When both ideas are used together, the improvement from Raw Attention is 0.240. So both things help, but their gains are not fully separate. Station 4 shows the same general pattern.

In Output 4.2, the error gets larger as the forecast goes further. So predicting step 48 is harder than predicting step 1. This increase is stronger on station 4. Period-routed ridge stays the lowest, while Raw Attention rises the most.

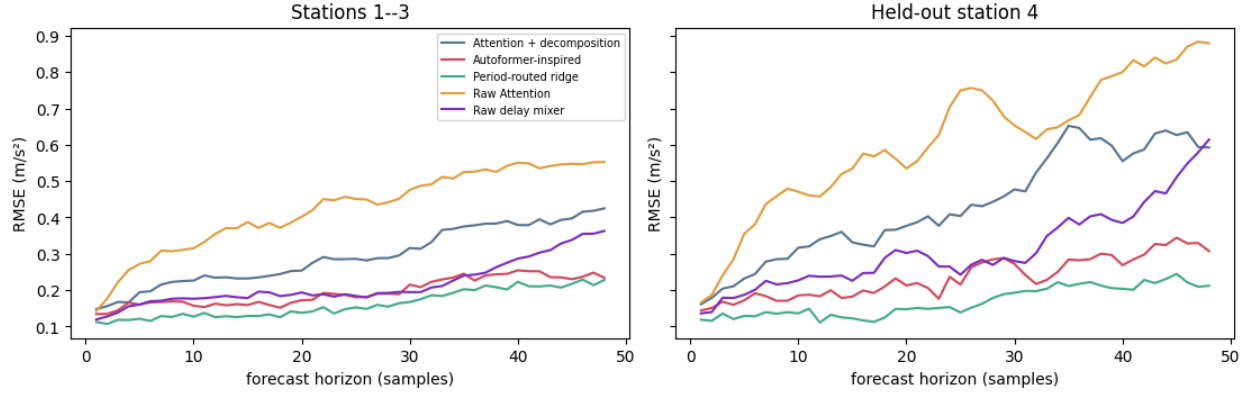


Figure 5: Output 4.2. RMSE over the 48 forecast steps.

For deployment, I would choose Period-routed ridge for both groups. It has the lowest validation error, only 59,904 parameters, and fit time is just 0.034 seconds. Autoformer-inspired is the best neural model, but its error is still higher, it has 373,026 parameters, and it takes around 192 seconds to fit. Sensor-specific ridge also cannot be used for station 4. On the final test data, Period-routed ridge gives RMSE 0.161 for stations 1–3 and 0.183 for station 4.

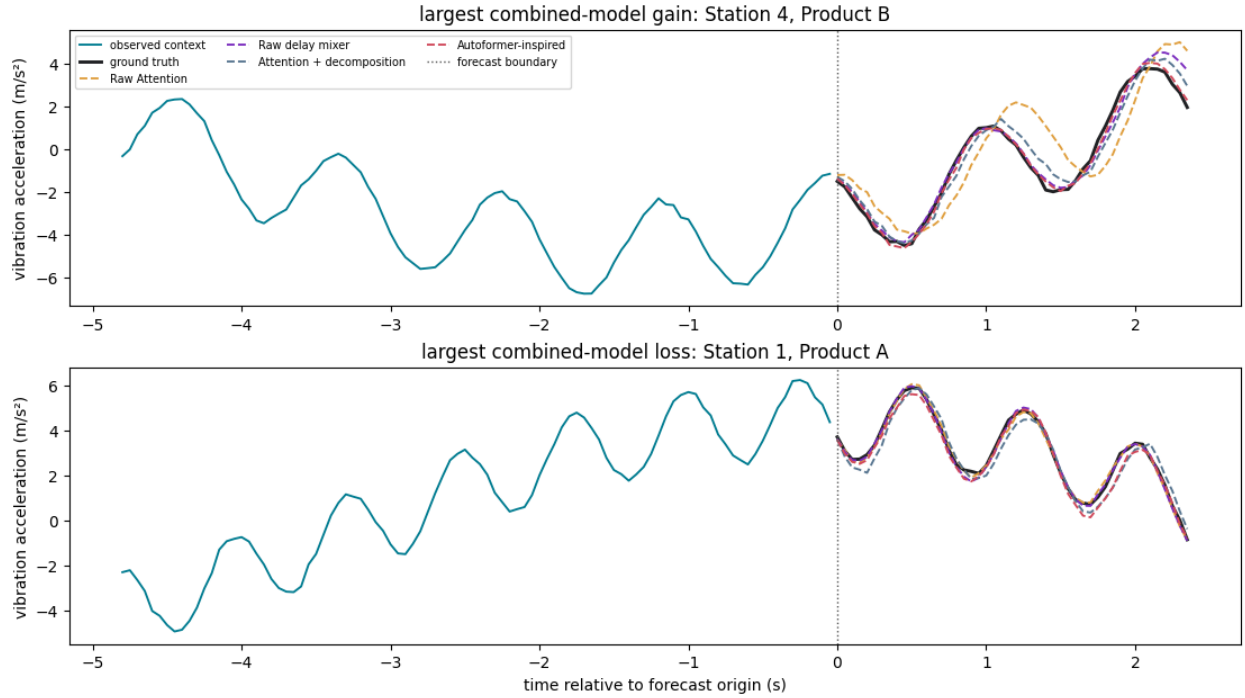


Figure 6: Output 4.1. Example where the combined model gains and where it loses.

Task 2

Question: Describe the preprocessing, the model, and the reasons for those choices. Compare the optional external file on and off, on more than one seed, using a time-ordered validation split. State the parameter count and epoch count submitted to the leaderboard.

Answer: So in Task 2, there are 43,656 old values and I have to predict the next 168 values, from time_idx 43657 to 43824. The target is never negative and its average is around 98. It can also change a lot inside one 168-step block. As a very simple baseline, copying only the last value gives RMSE 245.2 on the last four blocks. Copying the last 24 steps gives 175.5, which is better but still quite high.

For preprocessing, I put the past values on log scale. The mean and spread are calculated only from the data available at that stage, so future target values are not used by mistake. The six continuous external columns are scaled in the same way. The four on/off columns are already 0 or 1, so I left them like that.

The future external values for the next 168 steps are also given to the model because these are not target values. I also made a 14-day average for each hour of the day. I used this because one unusual last day should not decide the shape of the whole next week.

The final model is Autoformer. I used moving-average width 25 to split the window into a smooth part and a leftover part. Then FFT correlation is used to find the stronger repeating delays, and the leftover is mixed using shifted copies. So this part is delay mixing, not normal attention.

The model also reads the future external values and the hour-of-day pattern for the main estimate, and then the neural network adds a correction. I used Softplus at the end because the target cannot go below zero. The context length is 336, model width is 32 and there are 2 layers. The training loss is raw squared error because the leaderboard is checking raw RMSE.

For validation, I used the last four separate blocks of 168 steps in time order. I also ran two different random seeds, because I wanted to check that the result was not only coming from one lucky seed.

External file	Seed	RMSE	MAE	sMAPE	Best epoch	Parameters
Off	0	96.65	79.31	101.43	8	54,663
Off	1	96.65	79.57	101.73	3	54,663
On	0	69.39	49.54	78.61	10	55,280
On	1	69.67	49.70	79.84	9	55,280

Table 3: Task 2 validation with external file on and off.

So the external file is clearly useful here. For both seeds, RMSE becomes better by around 27 points when I turn the external file on. Because of that, I used it in the final submission. I trained both seeds again and averaged their 168-step forecasts. The leaderboard submission reports 110,560 parameters and 20 epochs. The submission was at rank 38.